Como medir os seus agentes de IA

Só para o palestrante: o que dizer em cada slide, transições, termos para explicar, a revelação em três versões, plano B, checklist e perguntas prováveis.

---

1. Guia de fala

Fonte: https://github.com/MateusCastro2203/agent-evaluation

Guia de fala

Este guia é material do palestrante (também fica no repo, para quem quiser reapresentar a palestra); não entra no ebook. Ele segue os 40 slides de `palestra/slides.md`, na mesma ordem e com os mesmos tempos das notas do apresentador. Para cada slide: o que dizer (em 2 a 5 frases), a transição para o próximo e os termos que valem uma frase de explicação para quem é júnior.

> **Se só der para lembrar de uma coisa:** a palestra não depende do resultado do run. Qualquer que seja o veredito (B ganha, inconclusivo, A ganha), a lição é a mesma: uma execução é uma amostra, e 16 execuções por variante quase nunca bastam para separar duas configurações. O relatório diz quantas faltam. Isso é medir.

Valores do run (de `evals/promptfooconfig.yaml` e `resultados/ensaio/NOTAS.md`): 2 variantes × 8 tarefas × 2 repetições = 32 execuções, modelo `sonnet`, concorrência 4. No ensaio, o run inteiro levou **~3 min 28 s** de relógio (cada execução, de 12 s a 42 s). Ou seja: o run dispara no minuto 0 e termina muito antes da revelação; o relatório só é aberto no minuto 40, porque é ali que a palestra chega na leitura. Se você mudar repetições, concorrência ou modelo, rode o ensaio de novo e atualize os números deste guia.

Resultado do ensaio (use como exemplo, nunca como "o resultado de hoje"): A 12/16 (75%, Wilson 51% a 90%), B 16/16 (100%, 81% a 100%), pass^2 6/8 × 8/8, 3 fora do escopo (todos de A), 0 `erro_agente`. Veredito: **inconclusivo**; o relatório estima 27 execuções por variante para as barras pararem de se sobrepor (critério do veredito), com piso de 24 pelo teste z de duas proporções.

Antes de subir no palco

Na véspera

1. Rodar o run completo (`pnpm eval`, grava `resultados/ultimo.json`) e anotar o tempo de relógio real (no ensaio: ~3 min 28 s). **O `pnpm eval` sai com código 100 quando alguma execução não passou:** é o comportamento normal do promptfoo, o JSON é gravado do mesmo jeito.
2. Logo em seguida, antes de rodar qualquer outra coisa, gravar esse run como ensaio: `pnpm ensaio:gravar`. Ele copia `resultados/ultimo.json` para `resultados/ensaio/resultado.json` tirando caminhos pessoais (a pasta do projeto vira `.`, o home vira `~`, as pastas temporárias viram `<tmp>`) e já gera

`resultados/ensaio/relatorio.html`. **Atenção:** `pnpm eval:rapido` (e qualquer `pnpm eval`) sobrescreve `resultados/ultimo.json`; se rodar antes de gravar, o ensaio vira um run de 1 tarefa.

3. Rodar `pnpm relatorio:ensaio` e conferir que ele regenera `resultados/ensaio/relatorio.html`. Ele só lê e escreve arquivo (sem rede, sem modelo), então funciona offline. Esse é o plano B.
4. Ler o relatório do ensaio inteiro e decorar três coisas: o veredito, o `n` por variante e as "execuções necessárias por variante (critério do veredito)" (campo `execucoesNecessariasPorVariante`; no ensaio, 38). Se o run ao vivo falhar, é isso que você vai mostrar.
5. Conferir que a página de links (o QR dos slides 2 e 40) já aponta para o repositório publicado e para o ebook.

Checklist pré-palco (30 minutos antes)

- [ ] **`claude` logado:** abrir `claude` no terminal, na pasta do repo, e confirmar que ele entra direto, sem pedir login. O repo usa o login normal do Claude Code; não há chave de API em lugar nenhum.
- [ ] **`pnpm eval:rapido` OK no local:** roda 1 tarefa, 1 repetição. Se passar, rede, login e cota estão funcionando. Se falhar, você já sabe que vai de plano B.
- [ ] **Notebook na tomada.** O run usa CPU e rede por alguns minutos (no ensaio, ~3 min), e o notebook fica ligado a palestra inteira.
- [ ] **Sem modo de suspensão:** desligar a suspensão e o descanso de tela nas configurações; no macOS, deixar `caffeinate -dims` rodando num terminal à parte durante a palestra.
- [ ] **Terminal com fonte grande** (dá para ler do fundo da sala) e na pasta do repo, com `pnpm eval` já digitado, sem dar Enter.
- [ ] **`relatorio.html` do ensaio aberto numa aba do navegador:** `resultados/ensaio/relatorio.html`, gerado por `pnpm relatorio:ensaio`. O relatório é gravado na mesma pasta do JSON de entrada, então o run ao vivo (`pnpm relatorio` → `resultados/relatorio.html`) não sobrescreve o do ensaio.
- [ ] **`resultados/ensaio/resultado.json` é o run completo da véspera,** não o do `pnpm eval:rapido` de agora há pouco (o rápido só sobrescreve `resultados/ultimo.json`).
- [ ] **Notificações desligadas** (modo Não Perturbe) e Slack/e-mail fechados.
- [ ] **Apresentar pelo `palestra/apresentacao.html`** aberto no navegador (Chrome ou Edge), em tela cheia (tecla `f`). Seta para a direita ou espaço avançam **cada clique** (respostas das perguntas, barras, frases de efeito) e depois o slide; seta para a esquerda volta. A tecla `p` abre a visão do apresentador numa janela à parte, com as notas e o próximo slide. O HTML é autocontido: funciona sem internet.
- [ ] **Passar os slides uma vez no HTML** antes de subir, para conferir que os cliques aparecem na ordem (slides 7, 14, 15, 20, 21, 23, 27, 30, 32, 39; as perguntas não têm cliques).

- [ ] **PDF como reserva:** `palestra/apresentacao.pdf` aberto noutra janela. Ele mostra cada slide já completo (sem cliques); se o navegador travar, siga pelo PDF e revele as respostas falando.
- [ ] Água por perto. A palestra é de 50 minutos sem pausa.

Slide a slide

Como funcionam as perguntas. Os slides com um selo laranja em cima ("Mão para cima", "Façam a aposta", "Chute antes de eu mostrar") são perguntas para a plateia, respondidas em voz alta ou com a mão. Regra para todas: o slide de pergunta é **um slide só, sem cliques**: tudo o que aparece (opções, frase) já está na tela. Faça a pergunta, espere no máximo 20 segundos e conte as mãos por alto ("uns dois terços", "quase ninguém"). Nos slides "Chute antes de eu mostrar", a resposta fica no **slide seguinte**: só avance depois de ouvir a sala. Não espere unanimidade nem silêncio total: o objetivo é a sala acordar, não fazer pesquisa.

Como funcionam os cliques. No `apresentacao.html`, seta para a direita ou espaço mostram o próximo item do slide (fragmento) e, quando não há mais itens, passam para o próximo slide. Cada slide diz abaixo quantos cliques tem. Slides de pergunta e slides de resposta não têm cliques.

Slide 1 — Capa · [00:00–00:15]

Diga: *"Hoje a pergunta é: como saber se o seu agente de código está bom?"* e avance.

Slide 2 — Quem é Mateus Castro? · [00:15–00:45]

Diga, em 30 s no máximo: uma frase por bloco do slide. Onde trabalha e com o quê ("AI Engineer na Creditas, com foco em soluções de IA"), por onde já passou (Gol, Smiles, Neon, Dotz) e quem é fora do código.

Sobre o QR: *"Esse QR leva para a minha página de links, onde ficam o repositório, o ebook e os slides. Não precisa fotografar agora: ele volta no último slide."*

Não se demore aqui: o run dispara no slide 4. Ele leva poucos minutos, mas quanto antes começar, mais folga para um imprevisto (login, rede).

Slide 3 — Pergunta: quem mede? · [00:45–01:15] · sem cliques

Pergunte (as três opções já estão na tela): *"Quem aqui mede hoje se o agente de código está melhorando?"*

Leia a sala, opção por opção: *"Quem não mede?"* (mãos) → *"Quem vai no feeling?"* (mãos) → *"Quem tem uma métrica?"* (mãos). Levante a mão junto no "no feeling": você também já fez. Conte as mãos por alto; no total, até 20 s.

O que dizer em cada caso:

- **Quase ninguém tem métrica** (o esperado): *"Normal. É para isso que a gente está aqui."*
- **Muita gente tem métrica:** *"Ótimo. No fim vocês me dizem se a minha é melhor que a de vocês."* Se der, pergunte a uma pessoa qual métrica usa (uma resposta só, sem debate).
- **Ninguém levanta a mão em nada:** *"Tudo bem, a próxima vale aposta."* E siga.

Transição: *"E para não ficar só na teoria, eu vou fazer uma aposta agora."*

Slide 4 — A aposta de hoje · [01:15–02:15]

Faça: dar Enter no `pnpm eval` que já está digitado. Confirmar que começou (aparece o progresso do promptfoo) e minimizar o terminal. Não voltar nele até o minuto 40. No ensaio, o run terminou em ~3 min: ele vai estar pronto bem antes; o relatório espera a revelação.

Diga: *"Eu escrevi um `CLAUDE.md` para este projeto, com as convenções do código. Acho que ele ajuda. Mas 'acho' não é medida. Então o Claude Code vai trabalhar 32 vezes enquanto eu falo: 8 tarefas, duas vezes cada, sem o arquivo (variante A) e com o arquivo (variante B). No minuto 40 a gente vê."*

Termos:

- **`CLAUDE.md`:** um arquivo de texto na raiz do projeto que o Claude Code lê no começo de toda sessão. É onde você escreve as regras da casa: comando de teste, padrões, pastas que não pode mexer.
- **Variante:** uma das configurações que você está comparando. Aqui só muda uma coisa entre A e B: a existência do `CLAUDE.md`.

Se o `pnpm eval` não começar (erro de login ou rede na hora): não tente consertar no palco. Diga *"o agente decidiu participar do plano B hoje; no minuto 40 eu mostro o run de ontem, gravado com o mesmo comando"* e siga.

Slide 5 — Pergunta: a aposta · [02:15–03:00] · sem cliques

Pergunte (opções e a frase "a resposta sai no minuto 40" já estão na tela): *"O `CLAUDE.md` vai fazer diferença?"*

Leia a sala, opção por opção: *"Quem acha que o B, com `CLAUDE.md`, ganha?"* → *"Quem acha que dá empate?"* → *"Quem acha que o A, sem o arquivo, ganha?"*. Conte por alto e **guarde a proporção** (ex.: "uns 70% no B, uns 20% no empate, quase ninguém no A"): você volta nela no

slide 33. Até 20 s.

O que dizer em cada caso:

- **Quase todo mundo no B:** *"Vocês confiam no meu arquivo. Vamos ver se ele merece."*
- **Muita gente no empate:** *"Tem gente desconfiada. Bom sinal."*
- **Ninguém no A:** *"Ninguém no A? Guardem isso."*

Fechamento: aponte a frase na tela: *"A resposta sai no minuto 40."*

Transição: *"Enquanto ele trabalha, vamos ao problema."*

Slide 6 — Divisória "1 · O problema" · [03:00]

Passe direto. Diga só: *"Vibe check não é medida."*

Slide 7 — "Parece que melhorou" · [03:00–05:00] · 1 clique

Diga (apontando o fluxo): *"A receita é sempre a mesma: muda alguma coisa, pede uma tarefa, deu certo, 'melhorou!'. O que ficou de fora: a tarefa era fácil? E se pedisse de novo? Quanto custou? Mexeu onde não devia?"* Clique: *"E a mais esquecida: daria certo antes da mudança também?"*

Pergunta opcional (só se a sala estiver fria): *"Quem já trocou de modelo porque 'sentiu' que estava melhor?"* Levante a mão junto.

O tom não é de bronca: vibe check é o ponto de partida. O problema é parar nele.

Termo:

- **Vibe check:** avaliar "no olho", pela sensação de uso, sem número.

Transição: *"E tem um motivo técnico para uma tarefa só não dizer nada."*

Slide 8 — O agente não é determinístico · [05:00–06:00]

Diga: *"Mesmo prompt, mesmo repo, mesmo modelo, e o agente faz caminhos diferentes. O modelo sorteia cada pedaço da resposta, e o loop de ferramentas amplifica qualquer diferença pequena do começo. Isso é normal, não é bug. Então uma execução é uma amostra, não um resultado."*

Termos:

- **Determinístico:** mesma entrada, sempre a mesma saída. Agente não é.
- **Amostra:** um resultado tirado de algo que varia. Como jogar uma moeda uma vez: não diz se ela é viciada.

Transição: *"Então quanto diz uma execução? Chutem."*

Slide 9 — Chute: "1 de 1 é 100%. Confiam?" · [06:00–06:45] · sem cliques

Pergunte: *"Passou 1 de 1. É 100%. Quem confia?"* Mãos; até 15 s. Espere a sala; só então avance.

O que dizer em cada caso:

- **Muita gente confia:** *"Vamos ver o que a conta diz."*
- **Ninguém confia:** *"Desconfiança saudável. Quanto, então?"* (deixe alguém gritar um número).

Transição: *"Vamos ver o que a conta diz."*

Slide 10 — Resposta: 1 de 1 · [06:45–08:00] · sem cliques

Diga (as duas barras já estão na tela): *"Com 1 de 1, a taxa real pode estar entre 21% e 100%. E com 16 de 16, ainda é entre 81% e 100%. Isso é conta, não opinião: é o intervalo de Wilson, o mesmo que o relatório vai usar no fim."*

Termo:

- **Intervalo de confiança (95%):** a faixa de valores "verdadeiros" compatíveis com o que você viu. Explicação completa no slide 36; aqui basta "é a margem de erro".

Transição: *"Se uma execução não basta, o que eu meço, e quantas vezes?"*

Slide 11 — Divisória "2 · O que é 'efetivo'" · [08:00]

Diga: *"Efetivo não é um número. São seis."*

Slide 12 — As métricas de um agente de código · [08:00–10:30]

Vá linha por linha, devagar. É o slide mais "de levar para casa" do bloco.

Diga:

1. *"Sucesso: o teste escondido da tarefa passou. É o critério de feito."*
2. *"Custo e tokens, turnos e tempo: o próprio* `claude -p` *devolve esses campos no JSON final."*
3. *"Fora do escopo: mexeu onde não devia."*
4. *"Consistência: resolve sempre, não só às vezes. Já explico."*

Sobre custo: *"O valor em dólar é uma estimativa calculada pelo próprio cliente. Com assinatura, leia como 'tamanho do esforço', não como fatura."*

Termos:

- **Token:** o pedaço de texto que o modelo lê e escreve (mais ou menos uma sílaba ou palavra curta). Mais tokens = mais trabalho = mais custo.
- **Turno:** uma ida e volta entre o modelo e uma ferramenta (ler um arquivo, editar, rodar o teste).
- **Critério de feito:** a regra objetiva que diz se a tarefa foi cumprida. Aqui, um teste automático.
- **`claude -p`:** o Claude Code rodando sem tela interativa: recebe o pedido por argumento e devolve o resultado.

Slide 13 — Sucesso sozinho engana · [10:30–12:00]

Diga: *"Dois agentes com a mesma taxa podem ter custo muito diferente, ou um deles pode resolver mexendo onde não devia. Fora do escopo é separado de propósito: resolveu, mas editou o código congelado, não é sucesso."*

Transição: *"Antes de contar sucesso, eu preciso dizer o que conta como sucesso."*

Slide 14 — Um rótulo por execução · [12:00–13:30] · 1 clique

Diga: *"Cada execução termina com um rótulo só: passou, falhou, fora do escopo ou erro do agente."* Clique: *"Erro do agente é timeout, login caído, limite de turnos: é contado à parte, porque não diz nada sobre a tarefa. Isso vem direto do caso real que eu mostro no bloco 4."*

Termo:

- **Timeout:** o tempo máximo que a execução pode levar; passou disso, ela é cortada.

Slide 15 — pass@k × pass^k · [13:30–15:00] · 3 cliques

Diga: *"pass@k é 'passou em pelo menos uma de k tentativas'. pass^k é 'passou em todas'."*

Pergunte (antes de clicar): *"Quando k cresce, pass@k sobe ou desce?"* Deixe gritarem; até 10 s. Clique: **sobe** (mais chances de acertar uma). *"E o pass^k?"* Clique: **desce** (mais chances de errar uma).

- **Se alguém errar:** *"Pensa em k = 100: acertar pelo menos uma de 100 é fácil; acertar as 100 é difícil."*

Clique final: *"Para um agente que roda sozinho, o que importa é o pass^k: ninguém vai ficar pedindo de novo até dar certo. É a métrica honesta para automação."*

Hoje, k = 2: das 8 tarefas, quantas passaram nas duas repetições. A conta é conservadora: uma repetição com `erro_agente` conta como "não passou".

Termo:

- **pass@k / pass^k:** lê-se "pass at k" e "pass hat k". O segundo vem do τ-bench (Yao et al., 2024).

Transição: *"Métrica definida. Agora: como montar a prova."*

Slide 16 — Divisória "3 · Evals" · [15:00]

Diga: *"Eval é tarefa mais critério de feito."*

Termo: eval (de *evaluation*): um teste automático para um sistema com IA. Uma lista de casos com o resultado esperado; o sistema roda em todos e você mede quantos acertou.

Slide 17 — O conjunto de hoje: 8 tarefas · [15:00–17:00]

Diga: *"O app de laboratório é uma agenda de meetups, sintética. As tarefas estão escritas como issue de verdade: 'não deixe inscrever em evento lotado', e só. Cinco dependem de uma convenção que só o `CLAUDE.md` explica: o tipo de erro do domínio, dinheiro em centavos, o relógio do projeto, a pasta congelada, a auditoria. Três são controle: não dependem de convenção nenhuma."* A lista das 8 tarefas está nas notas do slide, se alguém pedir.

Por que ter controle: *"Se o B ganhar também nas tarefas de controle, desconfie: é ruído ou outra causa. O efeito esperado está nas tarefas 03 a 07."*

Por que o prompt não cita a convenção: se dissesse "use `ErroDominio`", A e B empatariam e a gente não mediria nada.

Termos:

- **Tarefa de controle:** caso em que a mudança não deveria fazer diferença. Serve para checar se a diferença que aparece é mesmo da mudança.
- **Convenção:** regra do projeto que não está escrita no código; um dev novo no time só descobre lendo a documentação ou levando um comentário no code review.

Transição: *"Vamos abrir uma delas."*

Slide 18 — Uma tarefa por dentro · [17:00–19:00]

Diga: *"Em cima, o pedaço que importa do prompt da tarefa 03. É só isso que o agente vê. Embaixo, as asserções do teste escondido, que ele não vê: não basta lançar um erro, tem que ser o erro do domínio, o* `ErroDominio`*, com o código no formato da casa, maiúsculas com sublinhado, falando de lotação. É exatamente o que um dev novo erraria sem ler a documentação."*

Se perguntarem "por que não exigir um código exato?": *"Porque o prompt não diz qual código usar. Exigir um nome letra por letra mediria sorte, não convenção. O teste aceita qualquer nome razoável de lotação e só recusa reaproveitar* `CAPACIDADE_INVALIDA`*, que já existe e significa outra coisa."*

Diga: *"O teste nunca entra na pasta do agente. A correção roda numa pasta nova, com o app intacto e só os arquivos que o agente mudou, e o teste escondido entra por último, por cima de qualquer arquivo com o mesmo nome. Se o agente criar um 'teste oculto' falso para passar, não adianta."*

Termos:

- **Teste escondido (oculto):** o teste que decide se a tarefa passou, guardado fora do alcance do agente, como o gabarito de uma prova.
- **Escopo:** a lista de arquivos que a tarefa permite alterar (o campo `permitidos` do `tarefa.json`).
- **SCREAMING_SNAKE:** nome todo em maiúsculas, com sublinhado entre as palavras (ex.: `INSCRICAO_DUPLICADA`).

Transição: *"E como uma execução dessas roda de ponta a ponta?"*

Slide 19 — Como uma execução roda · [19:00–21:00]

Diga: *"Três passos. Um: cópia limpa do app numa pasta temporária; a variante B ganha o* `CLAUDE.md`*. O git que mede o que mudou fica fora da cópia, onde o agente não alcança. Dois: o Claude Code roda em modo headless, com saída JSON, só as ferramentas de uma lista fechada, dentro do sandbox do sistema operacional (sem rede, escrevendo só na cópia), com limite de turnos e timeout. Três: numa pasta nova, o app intacto mais o que o agente mudou mais o teste escondido; o Vitest roda num sandbox também, e o que mudou é comparado com a lista de permitidos."*

Diga: *"O promptfoo é o orquestrador: chama o provider, repete, roda em paralelo e grava o JSON. A métrica e o veredito ficam num script nosso."*

Isolamento (fale se houver sêniores na sala): *"O runner restringe a configuração ao projeto, com* `--setting-sources project`*, e desliga os conectores MCP da conta, com* `--strict-mcp-config`*. Assim o meu* `CLAUDE.md` *pessoal, os meus hooks e as minhas integrações não vazam para a variante A. Se vazassem, A teria um* `CLAUDE.md` *escondido e o experimento estaria contaminado. Eu*

verifiquei: perguntei para a variante A que instruções ela recebeu, e ela respondeu 'nenhuma'." Se alguém insistir: o que **não** dá para tirar são as configurações gerenciadas pela organização; elas chegam às duas variantes, igualmente.

As flags, se perguntarem (de `src/claude.ts`): `-p <prompt> --output-format json --model sonnet --max-turns 30`; `--permission-mode dontAsk` com `--allowedTools` (ler e editar só dentro da cópia, `pnpm test`, `pnpm vitest` e comandos de leitura: ls, cat, grep, head, wc); `--disallowedTools` (WebFetch, WebSearch, as deps do lab, o projeto e `~/.claude`); sandbox do SO por `--settings`; `--setting-sources project`; `--strict-mcp-config`; `--no-session-persistence`. Timeout de 10 minutos por execução.

Login: *"Usa a assinatura normal do Claude Code. Não precisa de chave de API."*

Termos:

- **Headless:** sem tela interativa. O agente recebe o pedido por argumento e devolve o resultado.
- **Provider:** no promptfoo, o pedaço de código que "responde" a cada caso. Aqui, ele roda o Claude Code numa cópia do repo.
- **Diff:** a lista do que mudou nos arquivos em relação à cópia original (calculada pelo git).
- **Vitest:** o executor de testes do projeto (como o Jest).
- **promptfoo:** ferramenta open source para rodar evals a partir de um arquivo de configuração.
- **Sandbox do SO:** uma "caixa" do sistema operacional que limita o que o processo pode ler, escrever e acessar na rede (no macOS, o Seatbelt).
- **MCP:** o protocolo que liga o agente a ferramentas externas (Slack, e-mail, banco). Aqui fica desligado.

Transição: *"Mas quem garante que a régua mede certo?"*

Slide 20 — Duas camadas de eval · [21:00–23:00] · 1 clique

Diga: *"Camada 1: testes do medidor, com um agente falso. Rodam offline, em segundos, a cada commit. Provam que a régua mede certo. Camada 2: o Claude Code de verdade nas 8 tarefas. Leva minutos, gasta cota, e roda quando algo muda: o prompt, o `CLAUDE.md`, o modelo."* Clique: *"Sem a camada 1, um bug no medidor parece 'o agente piorou'."*

Exemplos da camada 1: agente falso que não faz nada → `falhou`; que aplica a solução de referência → `passou`; que edita o código congelado → `fora_do_escopo`; que dá timeout → `erro_agente`, sem derrubar o run.

Bônus: toda tarefa precisa **falhar** no app intacto e **passar** com a solução de referência. Tarefa que já passa sem fazer nada é inútil; tarefa que nem a solução passa é impossível.

Termos:

- **Agente falso (mock):** um código que finge ser o agente e faz sempre a mesma coisa. Serve para testar o medidor sem gastar modelo.
- **Solução de referência:** a resposta certa, escrita por uma pessoa, para provar que a tarefa é possível.

Transição: *"E quando não dá para verificar com teste?"*

Slide 21 — LLM-as-judge · [23:00–25:00] · 1 clique

Diga: *"LLM-as-judge é usar um modelo para dar nota à saída de outro. Ajuda quando o critério não cabe num teste: clareza, estilo, mensagem de commit. É rápido de escrever. Mas tem vieses conhecidos (posição, tamanho da resposta, preferir o próprio estilo), e um juiz não determinístico avaliando um agente não determinístico soma variância."* Clique: *"Se dá para verificar com código, verifique com código."* Hoje, nenhuma tarefa precisa de juiz: o critério é 100% teste.

Pergunte (opcional, se o tempo deixar): *"Que tarefa do seu dia a dia não dá para verificar com teste?"* Pegue uma ou duas respostas, até 20 s. Respostas esperadas: "a documentação ficou clara?", "o PR está bem descrito?", "a mensagem de erro é útil?". Resposta: *"Ali o juiz pode valer, desde que você meça o juiz também: compare com nota humana numa amostra."*

Termos:

- **Viés de posição:** ao comparar duas respostas, o juiz tende a preferir a primeira (ou a segunda).
- **Viés de verbosidade:** tende a preferir a resposta mais longa.
- **Rubrica:** o critério escrito que o juiz segue (no promptfoo, `llm-rubric`).

Transição: *"Isso não é teoria de laboratório. Vou mostrar o que a gente aprendeu fazendo isso em produção."*

Slide 22 — Divisória "4 · Caso real: agentes KTLO na Creditas" · [25:00]

Diga: *"Um time da Creditas construiu agentes que resolvem chamados operacionais de suporte, o chamado 'keep the lights on'. Não vou mostrar números internos nem nomes de sistemas: só o que aprendemos sobre medir. São sete lições."*

Termo: KTLO (*keep the lights on*, "manter as luzes acesas"): o trabalho de manter o que já existe funcionando (chamados, correções, rotinas), em oposição a construir coisa nova.

Cuidado durante todo o bloco: não citar números, nomes de pessoas, nomes de sistemas ou parceiros, chaves de ticket nem achados de segurança, nem de memória, nem "por alto". Se perguntarem, use a resposta da pergunta 7 da seção de perguntas.

Slide 23 — O que conta como "deu certo" · [25:00–28:00] · 1 clique

Diga (lição 1): *"Toda execução termina com um rótulo: resolvido, assistido, escalado ou falta informação. 'Funcionou?' não é pergunta que se responde num painel."*

Clique (lição 2): *"Se um caso que nem era do escopo do agente entra na conta como falha, a taxa cai, e ninguém sabe se o agente piorou ou se entrou outro tipo de caso. Defina o desfecho antes de olhar o número."*

Ligação com o repo: *"É por isso que o repo de hoje tem quatro rótulos, e erro do agente fica à parte."*

Termos:

- **Assistido:** o agente diagnosticou, mas uma pessoa executou. Contar como "resolvido" infla o número; contar como "falha" esconde valor.
- **Escalado:** o agente passou o caso para uma pessoa porque não era para ele resolver.
- **Denominador:** o "de quantos" da taxa. Se ele muda de composição, a taxa muda sem o agente mudar.

Transição: *"A segunda dor foi a variação."*

Slide 24 — Chute: "Pare depois de N chamadas" · [28:00–28:45] · sem cliques

Contexto (diga antes): *"O custo variava muito de uma execução para outra, por causa dos turnos. A primeira ideia foi colocar no prompt: 'pare depois de N chamadas'."*

Pergunte: *"Quem acha que funciona?"* Mãos; até 15 s. Espere a sala; só então avance.

O que dizer em cada caso:

- **Muita gente acha que sim:** *"Eu também achava."*
- **Ninguém acha:** *"Vocês já apanharam disso, né?"*

Transição: *"Vamos ver."*

Slide 25 — Resposta: teto de turnos · [28:45–29:30] · sem cliques

Diga (as duas frases já estão na tela): *"Não segurou: o modelo não conta os próprios turnos. O teto de orçamento virou um hook, que é código e não tem como ser ignorado."* (Lição 3.)

Se perguntarem "quanto variava?": *"O suficiente para uma média esconder tudo. Por isso a gente passou a olhar mediana e cauda, e travou o teto em código."* Não dê número.

Termos:

- **Variância:** o quanto os resultados se espalham em torno da média. Alta variância = cada execução é uma surpresa.
- **Hook:** um script que o Claude Code roda antes ou depois de cada chamada de ferramenta. Se ele sair com código 2, a chamada é bloqueada, não importa o que o modelo queira.
- **Mediana / cauda:** a mediana é o valor do meio (não é puxada por extremos); a cauda são os casos extremos, os mais caros.

Slide 26 — Lá, o juiz foi rejeitado · [29:30–30:30]

Diga (lição 6): *"E ali o LLM-as-judge foi rejeitado: juiz não determinístico avaliando agente não determinístico soma variância. O critério ficou em código e em gabarito humano. Isso é ponto de discussão, não regra universal: no contexto de vocês pode compensar."*

Transição: *"As três últimas lições são sobre fazer a eval ficar melhor com o tempo."*

Slide 27 — Eval que só melhora · [30:30–33:00] · 3 cliques (um por linha)

Diga (lição 4): *"Duas camadas, igual ao slide 20: determinística a cada commit e modelo real contra um gabarito curado por humano. E o gabarito nasce vazio: a pessoa escreve a resposta esperada sem olhar o que o agente respondeu. Se você copia a resposta do agente para o gabarito, mede o agente contra ele mesmo."*

Diga (lição 5): *"Matriz-catraca: rota vezes desfecho. A lista de células sem cobertura só pode encolher; um teste quebra se alguém tirar cobertura."*

Diga (lição 7): *"'O caso que surpreendeu na terça é um teste na quarta.' E um aprendizado só vira regra quando aparece em dois ou mais casos, para não transformar uma exceção em regra para todo mundo."*

Termos:

- **Gabarito:** o resultado esperado de cada caso, escrito por uma pessoa.
- **Catraca:** mecanismo que só gira para um lado. Aplicado a eval: a cobertura só cresce.
- **Rota:** o tipo de caso que o agente recebe (o "caminho" que ele segue). A matriz cruza cada tipo com cada desfecho possível.
- **Cobertura:** ter pelo menos um caso de eval para aquela combinação.

Transição: *"Então, como eu uso tudo isso para melhorar o meu agente?"*

Slide 28 — Divisória "5 · Como melhorar" · [33:00]

Diga: *"Uma mudança por vez."*

Slide 29 — Uma mudança por vez · [33:00–35:00]

Diga: *"Rode a eval antes, para ter a linha de base. Mude uma coisa: o `CLAUDE.md`, uma skill, um hook, o modelo ou o prompt. Rode de novo, com repetições. Compare com intervalo, não com a média. Se você mudou o modelo e o `CLAUDE.md` juntos, não sabe quem ganhou."*

Dica prática: *"Commite a configuração do agente junto com o resultado da eval. Daqui a um mês você sabe qual versão gerou qual número."*

Termos:

- **Linha de base (baseline):** a medida de antes da mudança, para comparar.
- **Teste A/B:** comparar duas versões que diferem em uma coisa só. O experimento de hoje é um.

Transição: *"E onde mora cada tipo de regra?"*

Slide 30 — Onde cada regra mora · [35:00–37:00] · 1 clique

Diga: *"`CLAUDE.md` entra em toda sessão: bom para convenção do repo, comando de teste, pasta proibida. Mas é contexto, não obrigação: a própria documentação do Claude Code diz isso. Skill entra só quando é usada: boa para procedimento longo e material de referência, que custa quase nada até ser necessário; se a descrição for ruim, nunca é chamada. Hook roda antes ou depois de cada ferramenta: é para a regra que precisa valer sempre, e é código, então se testa como código."*

Clique: *"Regra que não pode falhar não mora no prompt. Mora no hook, no teste ou no CI."*

Ligação com o repo: *"Cada uma dessas é uma variante B possível: troque o `CLAUDE.md` por uma skill ou um hook e rode de novo."*

Termos:

- **Skill:** um pacote de instruções (e às vezes scripts) que o agente carrega quando a tarefa pede. Se a descrição for ruim, ele nunca é chamado.
- **Contexto:** tudo o que o modelo lê antes de responder. Contexto influencia, mas não obriga.

Transição: *"E o último hábito: transformar erro em teste."*

Slide 31 — Falha vira tarefa nova · [37:00–38:30]

Diga: *"O agente erra alguma coisa no dia a dia. Você escreve uma tarefa com o prompt real e um teste escondido que prova o certo. Confere que falha no app intacto e passa com a solução. E ela entra no conjunto e não sai mais. É a lição 7 do KTLO aplicada ao agente de código."*

Slide 32 — O portão no CI · [38:30–40:00] · 3 cliques

Diga (um clique por linha): *"O portão roda no CI. Inconclusivo passa."* Clique: *"Se A ficar melhor que B, com intervalos sem sobreposição, sai com código 1 e quebra o build."* Clique: *"Ele só quebra com evidência de piora."*

Por que o portão não olha a taxa crua: com 16 execuções, a taxa oscila de uma rodada para outra só por ruído; um portão "não pode cair" ficaria vermelho sem o agente ter piorado.

Por que inconclusivo não quebra: quebrar o build por falta de dado ensina o time a ignorar o portão.

Termos:

- **CI (integração contínua):** o pipeline que roda os testes a cada push ou PR (ex.: GitHub Actions).
- **Quebrar o build:** o pipeline fica vermelho e o merge é bloqueado.
- **Código de saída 1:** o jeito de um programa dizer "deu errado" para o terminal ou para o CI. Zero é sucesso.
- **Portão de qualidade:** regra automática que aprova ou barra uma mudança.

Transição: *"Já deu tempo. Vamos ver o que o agente fez."*

Slide 33 — Divisória "6 · Revelação" (volta à aposta) · [40:00–40:30]

Diga (volta à aposta do slide 5): *"No começo, [a proporção que você guardou, ex.: uns 70%] apostou no B, [x] no empate e [quase ninguém] no A. Vamos ver."* Não peça mãos de novo; só lembre.

Faça: voltar ao terminal e olhar o promptfoo:

- **Terminou:** rodar `pnpm relatorio` (lê `resultados/ultimo.json`). **Antes de abrir o HTML, confira na tabela do terminal a linha "n (sem erro do agente)": tem que dar 16 para cada variante.** Se der outro número (por exemplo 1, porque o `ultimo.json` é de um `pnpm eval:rapido` antigo, ou bem menos de 16 porque o run caiu no meio), não mostre: vá para o plano B. Com 16 e 16, abra `resultados/relatorio.html` no navegador (`open resultados/relatorio.html` no macOS). Se o `pnpm eval` mostrou código 100 no fim, é normal: quer dizer que alguma execução não passou.
- **Não terminou, ou caiu:** plano B (seção abaixo). Não espere no palco.

Slide 34 — Ao vivo: o relatório · [40:30–43:00]

Diga: *"Eu vou ler numa ordem, porque a ordem importa."*

1. **n por variante:** *"Quantas execuções cada lado teve de verdade, e quantas foram erro do agente, contadas à parte."* Se o `n` de uma variante ficou bem abaixo de 16, diga: *"menos amostra, intervalo mais largo: guardem isso."*
2. **Taxa com intervalo:** leia a taxa de A e a de B **com** a faixa. Nunca só a taxa.
3. **Veredito:** leia em voz alta o que o relatório diz. Compare com a aposta da plateia: *"quem apostou no [B / empate / A]..."*. Inconclusivo não é empate; se a maioria apostou em empate, diga: *"quase: é 'não sei ainda', não 'são iguais'."*
4. **pass^k:** *"Quantas tarefas passaram nas duas repetições."*
5. **Fora do escopo:** *"Alguém resolveu mexendo onde não devia?"*
6. **Por tarefa:** *"O ganho, se houver, está nas tarefas de convenção (03 a 07) ou nas de controle?"*

Não comemore nem lamente ainda: *"Os próximos slides explicam como ler isso sem se enganar."* A fala da revelação, já adaptada ao resultado, está na seção "A revelação em 3 versões". Escolha a versão aqui, olhando o veredito.

Slide 35 — Chute: "B é o dobro de A. B ganhou?" · [43:00–43:45] · sem cliques

Pergunte: *"Exemplo inventado, não é o de hoje: A passou 6 de 16, B passou 12 de 16. B é o dobro. Quem acha que B ganhou?"* Mãos; até 15 s. Espere a sala; só então avance.

O que dizer em cada caso:

- **A maioria disse "ganhou":** *"É a resposta natural. E é exatamente o 'parece que melhorou' do começo."*
- **A maioria disse "não":** *"Vocês já pegaram o jeito. Agora a regra exata."*

Transição: *"Vamos ver o que a conta diz."*

Slide 36 — Resposta: 6 de 16 × 12 de 16 · [43:45–45:00] · sem cliques

Diga (barras e veredito já estão na tela): *"A: 37,5%, com intervalo de 18% a 61%. B: 75%, de 51% a 90%. As barras se encostam entre 51% e 61%. Então: inconclusivo."*

Diga a regra do repo: *"Se o limite inferior de B passa do limite superior de A, 'B melhor'. O contrário, 'A melhor'. Senão, inconclusivo. E inconclusivo não é empate: é 'não tenho dado para dizer'."*

Se um sênior perguntar "por que não um teste de hipótese?": *"Um teste z de duas proporções chamaria 6/16 contra 12/16 de significativo, p perto de 0,03. Barras sem sobreposição é um critério mais exigente, de propósito: o repo prefere não declarar vencedor cedo."*

Por que Wilson e não a conta simples (`p ± 1,96·√(p(1−p)/n)`): com `n` pequeno ou taxa perto de 0% ou 100%, a conta simples sai de [0, 1] ou dá largura zero (16 de 16 daria "100% ± 0"). Wilson não tem esse problema.

Termos:

- **Intervalo de Wilson:** um jeito de calcular a margem de erro de uma taxa (acertos ÷ tentativas) que funciona bem com poucas amostras. Proposto por Wilson em 1927.
- **Limite inferior / superior:** as pontas da barra.
- **Sobreposição:** as duas barras dividem um trecho. Se dividem, os dados são compatíveis com A e B iguais.

Transição: *"Então, quantas execuções eu precisaria?"*

Slide 37 — Chute: quantas execuções? · [45:00–45:30] · sem cliques

Pergunte (opções já na tela): *"Quantas execuções por variante para separar A de B? Quem diz 5? 16? 40? 200?"* Mãos em cada uma; até 20 s no total. Espere a sala; só então avance.

Slide 38 — Resposta: 27 por variante · [45:30–46:00] · sem cliques

Diga: *"No ensaio, com A 12 de 16 e B 16 de 16, o relatório diz 27 por variante."* Diga **"do ensaio"** em voz alta: é o run gravado, não o de hoje. Se o relatório de hoje mostrou outro número em "execuções necessárias por variante (critério do veredito)" (campo `execucoesNecessariasPorVariante`), complete: *"no de hoje, deu [N]"*. Se as duas taxas forem iguais, o relatório diz "não calculável".

O que dizer em cada caso:

- **Muita gente no 16:** *"Chegaram perto, mas é o que a gente rodou hoje, e não bastou."*
- **Muita gente no 40:** *"Um pouco a mais do que precisava."*
- **Muita gente no 200:** *"Para essa diferença, menos. Mas guardem o 200: ele volta no próximo slide."*

Slide 39 — Por que 16 não bastam · [46:00–47:00] · 1 clique

Diga: *"Se as taxas continuassem as mesmas, quantas execuções por variante até as barras pararem de se sobrepor? Para 37,5% contra 75%, 27. Para 62,5% contra 75%, uma diferença mais realista, 212."* Clique: *"Diferença pequena, muitas execuções. O número cresce com o inverso do quadrado da diferença."*

Explique o parêntese, se alguém olhar: é o piso pelo teste z de duas proporções, a fórmula clássica de tamanho de amostra. Ele costuma ser menor, porque o teste z é menos exigente que "barras sem sobreposição". Com o piso, o relatório ainda pode sair inconclusivo; quem manda é o primeiro número. No ensaio: 27, com piso de 24.

Diga (fechamento, vale para qualquer desfecho): *"Eu não sei ainda se o meu `CLAUDE.md` presta. Mas agora eu sei quanto eu preciso rodar para saber. Isso é medir."* (Na versão "B ganha", adapte: veja a seção da revelação.)

Se perguntarem a conta: o número principal não tem fórmula fechada: o relatório testa n = 1, 2, 3… com as taxas de hoje e para no primeiro n em que os intervalos de Wilson não se sobrepõem. O piso vem da fórmula clássica para comparar duas proporções, `n ≈ (1,96 + 0,84)² · [pA(1−pA) + pB(1−pB)] / (pA − pB)²`: o 1,96 vem dos 95% de confiança; o 0,84, dos 80% de poder.

Termos:

- **Poder (estatístico):** a chance de o experimento detectar a diferença se ela existir de verdade. Com 80%: se o `CLAUDE.md` ajuda mesmo, em 8 de cada 10 experimentos desse tamanho a gente veria.
- **Estimativa, não garantia:** a conta usa as taxas observadas, que também têm erro.

Transição: *"Para fechar, o que levar para casa."*

Slide 40 — Leve para casa · [47:00–50:00]

Diga: *"Cinco hábitos. Três estão no slide: desfecho antes do número; repetir e olhar intervalo, não uma execução; toda falha vira tarefa nova. Os outros dois: tarefas reais com teste escondido e escopo, e uma mudança por vez. O repositório roda o experimento de hoje com o login do Claude Code, e o ebook em PDF está dentro dele."*

Faça: deixar o slide parado enquanto as pessoas fotografam o QR. Conte até cinco antes de falar de novo. Abra para perguntas **neste slide** (quem chegou atrasado ainda fotografa o QR). As respostas prováveis estão na seção "Perguntas prováveis".

Desafio para quem levar o repo: *"Troque o `CLAUDE.md` por uma skill ou um hook e rode de novo. Ou aponte as tarefas para o seu próprio projeto."*

Slide 41 — Referências (apêndice)

Só mostre se alguém pedir as fontes. **Diga, se mostrar:** *"Todas as referências são públicas e estão no ebook com link. O SWE-bench está aí porque é o mesmo formato, em escala de pesquisa: issues reais de GitHub como tarefa para agente de código."* Depois, volte ao slide 36.

A revelação em 3 versões

Escolha a versão no slide 34, olhando o veredito do relatório. As três contam a mesma história: **16 execuções por variante não bastam, e essa é a lição.** Use os números do relatório de hoje (ou do ensaio, no plano B); não arredonde nem "melhore" nenhum.

Regra do veredito (para não errar ao vivo):

- **B melhor:** o limite inferior de B é maior que o limite superior de A.
- **A melhor:** o limite inferior de A é maior que o limite superior de B. É o **único** caso em que o portão sai com código 1.
- **Inconclusivo:** qualquer outro caso (as barras se sobrepõem).

Versão 1 — B ganha

Abertura: *"O `CLAUDE.md` ganhou, e com barras que não se encostam. Agora vem a pergunta que importa: ganhou onde?"*

Leia: o resultado por tarefa. O ganho está nas tarefas de convenção (03 a 07)? Ótimo, é o efeito esperado. Está também nas de controle (01, 02, 08)? Diga: *"isso me deixa desconfiado: o `CLAUDE.md` não deveria mudar nada ali. Pode ser ruído, pode ser que ele ajude de um jeito que eu não previ. Vale rodar de novo."*

Lição: *"Para aparecer com 16 execuções, a diferença precisou ser grande. Se o meu `CLAUDE.md` tivesse ajudado só um pouco, o relatório estaria dizendo inconclusivo, e o efeito seria igualmente real. Uma diferença pequena precisaria de centenas de execuções (aponte o 212 do slide 39). E ganhar uma vez não é ganhar sempre: o próximo passo é repetir o experimento outro dia."*

Fechamento adaptado do slide 39: *"Desta vez 16 bastaram, porque a diferença era grande. Na maioria das mudanças que vocês vão fazer, não vai ser. Agora vocês sabem calcular quanto rodar."*

Versão 2 — Inconclusivo

Abertura: *"Inconclusivo. E eu não vou fingir que isso é empate."*

Leia: as duas taxas e as duas barras, e aponte o trecho em que elas se sobrepõem. Depois, as "execuções necessárias por variante (critério do veredito)" de hoje (no ensaio, 38).

Lição: *"Pode ser que o `CLAUDE.md` ajude, e pode ser que não. Com 16 execuções por variante, o dado não deixa dizer. Se eu tivesse rodado uma vez só, como todo mundo faz, eu teria saído daqui com uma certeza falsa, para um lado ou para o outro. O relatório diz que, se as taxas se mantiverem, eu precisaria de [N] execuções por variante para as barras pararem de se sobrepor. Esse número é o resultado mais útil de hoje."*

Sobre o portão: *"E reparem: o portão não ficou vermelho. Inconclusivo não quebra o build, porque falta de dado não é evidência de piora."*

Se B está na frente mas inconclusivo: *"B está na frente, mas 'na frente' com essas barras é exatamente o 'parece que melhorou' do começo da palestra. A diferença entre os dois é que agora eu sei o tamanho da minha dúvida."*

Versão 3 — A ganha

Abertura: *"A ganhou. O meu `CLAUDE.md` atrapalhou, e o portão teria barrado o meu próprio merge."*

Faça: mostrar o terminal. O `pnpm relatorio` saiu com código 1, e o pnpm imprime uma mensagem de erro: *"isso aqui não é bug: é o portão funcionando."*

Leia: por tarefa, onde A ganhou. Hipóteses para falar em voz alta (sem afirmar nenhuma antes de abrir as execuções): uma instrução do `CLAUDE.md` que confunde o agente numa tarefa de controle; mais contexto levando a mais turnos e a estourar o limite; uma regra que empurra o agente a mexer fora do escopo.

Lição: *"Eu escrevi esse arquivo achando que ajudava. Se eu tivesse confiado no vibe check, ele estaria no repositório agora, piorando o agente de todo mundo. Medir serviu exatamente para isso. E mesmo assim: são 16 execuções. A evidência é suficiente para não fazer o merge, não para concluir que todo `CLAUDE.md` atrapalha. O próximo passo é abrir as execuções que falharam e transformar o que eu vir numa tarefa nova."*

Casos de borda

- **Muitos `erro_agente`** (rede, cota, timeout): diga *"olhem o n: em vez de 16, a variante X teve [n]. Erro do agente fica à parte, e o intervalo ficou mais largo. É mais um motivo para não concluir nada com pouca amostra."* Se a maioria for erro, prefira mostrar o ensaio (plano B) e use o ao vivo só como exemplo de por que `erro_agente` é contado à parte.
- **Uma variante com n = 0:** o relatório roda mesmo assim e mostra inconclusivo. Vá de plano B.

Plano B: o run ao vivo falhou

Quando usar: no minuto 40, se o `pnpm eval` não terminou, caiu (rede, login, cota) ou terminou com quase tudo `erro_agente`.

Passo a passo:

1. **Não mate o processo no palco** e não tente consertar. Minimize o terminal.
2. Vá para a aba do navegador com o relatório do ensaio (`resultados/ensaio/relatorio.html`, aberta no checklist).
3. **Se a aba sumiu:** num terminal novo, na pasta do repo, rode `pnpm relatorio:ensaio`. Ele lê `resultados/ensaio/resultado.json` (o run da véspera, gravado com `pnpm ensaio:gravar`), funciona offline e grava `resultados/ensaio/relatorio.html`. Abra com

`open resultados/ensaio/relatorio.html` (macOS) ou arrastando o arquivo para o navegador.
4. Siga o slide 34 e a versão da revelação que corresponde ao veredito **do ensaio** (você decorou na véspera).

O que dizer:

> *"O run de hoje não terminou a tempo. Isso acontece com agente de verdade, e é exatamente por isso que existe um ensaio: o run completo de ontem, com o mesmo comando e as mesmas 32 execuções, gravado na pasta do ensaio. O relatório que eu vou mostrar é esse. E reparem: dá para rodar ele sem internet e sem assinatura. Quem levar o repositório pode fazer o mesmo."*

Se o run caiu por erro do agente, acrescente: *"E é por isso que erro do agente é um rótulo próprio: se eu contasse isso como falha da tarefa, o meu `CLAUDE.md` teria 'piorado' por causa do Wi-Fi."*

Se o run terminar durante as perguntas: pode mostrar o `pnpm relatorio` ao vivo no fim, como bônus: *"o de hoje acabou de sair; vamos ver se concorda com o de ontem."* Se não concordar, é mais uma prova da lição.

Perguntas prováveis

1. **"E o Codex? E o Gemini CLI?"** *"O método é o mesmo; só muda quem executa a tarefa. No repo, a parte que chama o agente fica isolada atrás de uma interface, a `RodarAgente`. Para medir outro agente, você implementa outra `RodarAgente` que chama a CLI dele em modo não interativo e devolve o mesmo formato de resultado. Tarefas, teste escondido, escopo e relatório continuam iguais. Isso está como desafio extra no repo. E não é um benchmark de qual ferramenta é melhor: é para você medir a configuração que você usa."*

2. **"LLM-as-judge serve para quê, então?"** *"Para o que não cabe num teste: clareza de documentação, qualidade da descrição do PR, se a mensagem de erro é útil. Ele é bom para triagem, antes do olho humano. A condição é medir o juiz também: pegar uma amostra, dar a nota você mesmo e ver se ele concorda. E nunca usar juiz para o que um teste consegue verificar."*

3. **"Quantas execuções eu preciso?"** *"Depende do tamanho da diferença que você quer enxergar. Uma diferença grande, tipo 37,5% contra 75%, pede umas 27 por variante até as barras se separarem. Uma diferença realista, 62,5% contra 75%, pede umas 212. O relatório do repo faz essa conta com os seus números e mostra como 'execuções necessárias por variante (critério do veredito)'; logo abaixo vem o piso pelo teste z de duas proporções, que é menor e não basta para o veredito. No ensaio, por exemplo: 38, com piso de 34. Comece pequeno, veja a estimativa e decida se vale rodar mais."*

4. **"Isso não fica caro?"** *"O repo usa a cota da assinatura do Claude Code, com o login normal. O relatório mostra o custo estimado por execução, que o próprio cliente calcula. Dá para economizar de três jeitos: a camada 1 (o medidor com agente falso) é offline e de graça, então roda a cada commit; a camada 2 roda só quando você muda alguma coisa no agente; e o* `pnpm eval:rapido` *roda uma tarefa só, para checar que está tudo de pé antes do run completo."* Não dê o custo do run de hoje sem ter o número do relatório na tela.

5. **"Preciso de chave de API?"** *"Não. O repo usa o login normal do Claude Code. E quem não tem assinatura consegue rodar o relatório sobre o run gravado (*`pnpm relatorio:ensaio`*), offline."* Não recomende chave de API nem como alternativa.

6. **"E se o meu projeto não tem teste?"** *"Então o primeiro entregável é o teste da tarefa. O critério de feito vem antes do agente: se você não consegue escrever um teste que diga se a tarefa foi cumprida, nem você sabe o que pediu. Comece com uma tarefa só, de um erro que o agente já cometeu."*

7. **"Quais foram os números do KTLO?"** *"São números internos que eu não posso compartilhar. O que eu posso passar são as lições e um método que vocês reproduzem no repo: lá está o run gravado do ensaio, com o JSON e o relatório, e vocês podem rodar o de vocês."* Não dê faixa, ordem de grandeza nem "mais ou menos". Se insistirem: *"não posso mesmo; mas o método está todo no repo, e o número que importa para vocês é o do agente de vocês."*

8. **"Por que só duas repetições?"** *"Porque duas repetições já mostram o ponto: elas dão 16 execuções por variante, e o relatório acabou de mostrar que é pouco. Para a palestra, eu quis um run curto e que coubesse folgado (no ensaio, levou uns 3 minutos) e um número que vocês conseguem conferir no repo. No seu projeto, sem plateia esperando, rode mais: é só mudar o* `repeat` *em* `evals/promptfooconfig.yaml`*."*

9. **"O agente não pode trapacear? Editar o teste?"** *"O teste escondido nunca entra na pasta do agente: a correção roda numa pasta nova, com o app intacto e só os arquivos que ele mudou, e o teste entra por último, por cima de qualquer arquivo com o mesmo nome. Se ele mexer em arquivo fora dos permitidos, criar link simbólico ou criar um* `.git`*, a execução vira* `fora_do_escopo`*. As dependências que rodam o teste são somente leitura e conferidas por hash; se mudarem, a execução vira* `erro_agente`*. E a configuração do agente fica restrita ao projeto, para o meu* `~/.claude` *pessoal não vazar para a variante A."*

10. **"Por que intervalos sem sobreposição? Isso não é conservador demais?"** *"É, de propósito. Um teste de hipótese clássico declararia vencedor antes. Para um portão de CI, eu prefiro errar para o lado de 'não sei': um portão que grita cedo demais ensina o time a ignorá-lo. Se você quiser outra regra, ela fica num script só, e dá para trocar."*

Se o tempo apertar: responda 1, 3 e 7 primeiro; são as que mais aparecem e a 7 é a que exige mais cuidado.

Glossário rápido

Termo	Em uma frase
Agente de código	um modelo que lê, edita e roda código sozinho, usando ferramentas, até cumprir uma tarefa
`CLAUDE.md`	arquivo de texto que o Claude Code lê no começo de toda sessão, com as regras do projeto
Variante	uma das configurações comparadas (A sem `CLAUDE.md`, B com)
Eval	teste automático de um sistema com IA: casos com resultado esperado
Critério de feito	a regra objetiva que diz se a tarefa foi cumprida (aqui, um teste)
Teste escondido	o teste que decide a tarefa, que o agente não vê
Escopo	os arquivos que a tarefa permite alterar
Tarefa de controle	caso em que a mudança não deveria fazer diferença
Token	pedaço de texto que o modelo lê ou escreve
Turno	uma ida e volta entre o modelo e uma ferramenta
Headless	rodar o agente sem tela interativa (`claude -p`)
Provider	no promptfoo, o código que responde a cada caso
promptfoo	ferramenta para rodar evals a partir de um arquivo de configuração
Diff	o que mudou nos arquivos em relação à cópia original
pass@k	passou em pelo menos uma de k tentativas
pass^k	passou em todas as k tentativas
Intervalo de Wilson	margem de erro de uma taxa que funciona bem com poucas amostras
Poder	chance de o experimento detectar uma diferença que existe
Inconclusivo	as barras se sobrepõem: o dado não deixa dizer quem é melhor
LLM-as-judge	um modelo dando nota à saída de outro
Hook	script que roda antes ou depois de cada ferramenta e pode bloquear a ação
Skill	pacote de instruções que o agente carrega só quando precisa
CI	pipeline que roda os testes a cada push ou PR

Termo	Em uma frase
Portão de qualidade	regra automática que aprova ou barra uma mudança
KTLO	*keep the lights on*: o trabalho de manter funcionando o que já existe
Gabarito	o resultado esperado de cada caso, escrito por uma pessoa
Catraca	mecanismo que só gira para um lado: a cobertura da eval só cresce